State Space Search Algorithms

Comprehensive Visual Cheatsheet

Machine Intelligence - Daniel Fišer - Fall 2025

Contents

1 State Space Fundamentals

1.1 State Space Definition

State Space - Mathematical Definition

A **state space** is a tuple:

$$\Theta = (\mathcal{S}, \mathcal{A}, \text{cost}, \mathcal{T}, I, \mathcal{S}_G)$$

where each component represents:

- $\mathcal{S}$ = finite set of **states**
  - Each state represents a configuration of the world
  - Example: In a puzzle, each arrangement is a state
- $\mathcal{A}$ = finite set of **actions**
  - Each action transforms one state into another
  - Example: "move tile left", "drive from A to B"
- **cost** : $\mathcal{A} \mapsto \mathbb{N}_0$ = **cost function**
  - Assigns a non-negative cost to each action
  - Example: distance traveled, time taken, energy used
- $\mathcal{T} \subseteq \mathcal{S} \times \mathcal{A} \times \mathcal{S}$ = **transition relation**
  - Defines which actions are applicable in which states
  - Must be **deterministic**: for any state s and action a, there is **at most one** resulting state s'
  - $(s, a, s') \in \mathcal{T}$ means: "applying action a to state s yields state s'"
  - Action a is **applicable** in s if such (s, a, s') exists
- $I \in \mathcal{S}$ = **initial state**
  - The starting point of the search
  - There is exactly one initial state
- $\mathcal{S}_G \subseteq \mathcal{S}$ = set of **goal states**
  - States we want to reach
  - There can be multiple goal states

1.2 Notation and Terminology

Transition Notation

Instead of writing $(s, a, s') \in \mathcal{T}$, we use the more intuitive notation:

$$s \xrightarrow{a} s'$$

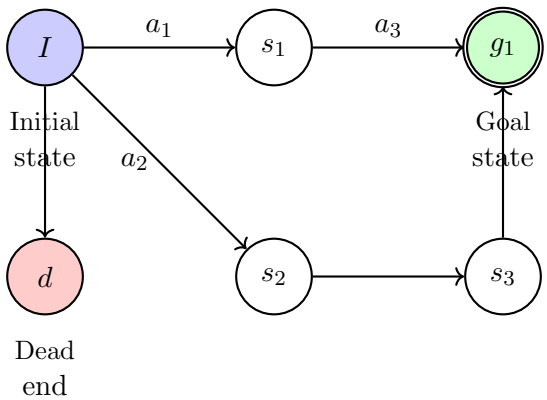
This reads as: "state s transitions to state s' via action a"
When the action is not important, we simply write: $s \to s'$

State Relationships

- s' is a **successor** of s if $s \xrightarrow{a} s'$ for some action a
  - Visual: $s \to s'$ (arrow points from s to s')
- s is a **predecessor** of s' if $s \xrightarrow{a} s'$ for some action a
  - Visual: $s \to s'$ (arrow originates from s)
- s' is **reachable from** s if there exists a sequence of transitions:

$$s = s_0 \to s_1 \to s_2 \to \cdots \to s_n = s'$$

- s is **reachable** (without reference) if s is reachable from I
- s is **solvable** if some goal state $s' \in \mathcal{S}_G$ is reachable from s
- s is a **dead end** if no goal state is reachable from s



1.3 Paths

Path Types

A **path** is a sequence of transitions from one state to another:

$$\tau = ((s_0, a_1, s_1), (s_1, a_2, s_2), \ldots, (s_{n-1}, a_n, s_n))$$

We distinguish three types of paths:

1. **Transition-path**: τ itself (includes states and actions)
2. **Action-path**: $\pi = (a_1, a_2, \ldots, a_n)$ (just the actions)
3. **State-path**: $\sigma = (s_0, s_1, \ldots, s_n)$ (just the states)

Alternative notation:

$$\tau = s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} \cdots \xrightarrow{a_n} s_n$$

Cost of a path:

$$\text{cost}(\tau) = \text{cost}(\pi) = \sum_{i=1}^{n} \text{cost}(a_i)$$

Special case: Empty path ($n = 0$) is valid. Every state s is reachable from itself with cost 0.

Path Example

Consider the path: $A \xrightarrow{drive} B \xrightarrow{load} B \xrightarrow{drive} C$

- **Transition-path**: $((A, drive, B), (B, load, B), (B, drive, C))$
- **Action-path**: $(drive, load, drive)$
- **State-path**: (A, B, B, C)
- **Cost**: If $\text{cost}(drive) = 5$ and $\text{cost}(load) = 1$, then $\text{cost}(\tau) = 5 + 1 + 5 = 11$

1.4 Solutions

Solution Concepts

- A **solution for state** s is a path τ from s to some goal state $s' \in \mathcal{S}_G$
- A **solution for** Θ (or simply "solution") is a solution for the initial state I
- An **optimal solution** is a solution with **minimal cost**
- Θ is **solvable** if a solution exists, otherwise **unsolvable**

1.5 Cost Notation

Important Cost Values

- $g(n)$ = path cost from initial state to node n
  - This is the actual cost incurred so far
  - Computed incrementally: $g(n') = g(n) + \text{cost}(a)$ where n is parent of n'
- $g^*(s)$ = optimal cost to reach state s from I
  - This is the **minimum possible** cost to reach s
  - We usually don't know this until we find it!
- g^* = cost of an optimal solution (from I to any goal)
- An action a is **unit cost** if $\text{cost}(a) = 1$
- Θ is **unit cost** if all actions have cost 1

1.6 Search Node Structure

Search Node

A **search node** n is different from a state! It contains:

Search Node n

n.State: The state this node represents

n.Pred: Pointer to parent node (or `null`)

n.Action: Action that led to this node (or `null`)

g(n): Cost of path from I to this node

Key distinction: The same state can appear in multiple search nodes!
Example: In tree search, if we reach the same state via two different paths, we create two different nodes.

1.7 Search Terminology

Key Search Terms

- **Node expansion**: Generating all successors of a node
  - Apply all applicable actions to the node's state
  - Create a new search node for each resulting state
  - After expansion, the state is said to be expanded
- **Open list (frontier)**: Set of nodes waiting to be expanded
  - These are candidates for the next expansion
  - Different algorithms use different data structures (stack, queue, priority queue)
- **Closed list (explored set)**: Set of states already expanded
  - Only used in graph search
  - Prevents re-expanding the same state
- **Tree search**: Search without duplicate elimination
  - No closed list
  - Same state can be expanded multiple times
- **Graph search**: Search with duplicate elimination
  - Maintains closed list
  - Each state expanded at most once

2 Blackbox API Functions

State Space API

When we solve problems, we typically implement these functions:

- `InitialState()`: Returns the initial state I
- `GoalTest(s)`: Returns `true` if and only if $s \in \mathcal{S}_G$
- `Cost(a)`: Returns the cost of action a
- `Actions(s)`: Returns the set of actions applicable in state s
- `Successor(s, a)`: Returns the state s' where $s \xrightarrow{a} s'$
  - Requires that action a is applicable in s

2.1 Helper Functions for Search Algorithms

InitNode() - Create Initial Search Node

Purpose: Create the root node of the search tree

```
function InitNode
    n.State ← InitialState()          ▷ Get the starting state
    n.Pred ← null                     ▷ No parent for root node
    n.Action ← null                   ▷ No action led to root
    g(n) ← 0                          ▷ No cost to reach starting point
    return n
end function
```

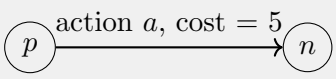
SuccNode(p

Purpose: Create a new node by applying action a to parent node p
Input:

- p = parent node
- a = action to apply

```
function SuccNode(p, a)
    n.State ← Successor(p.State, a)   ▷ Apply action to get new state
    n.Pred ← p                        ▷ Record parent for path reconstruction
    n.Action ← a                      ▷ Record which action was used
    g(n) ← g(p) + Cost(a)             ▷ Accumulate cost!
    return n
end function
```

Visual:



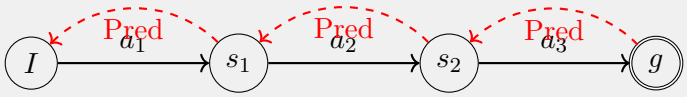
$g(p) = 10$ $\qquad$ $g(n) = 10 + 5 = 15$

ExtractSolution(n) - Reconstruct Path

Purpose: Given a goal node, reconstruct the action-path from initial state
Method: Follow predecessor pointers backward, collecting actions

```
function ExtractSolution(n)
    π ← ()                                  ▷ Initialize empty action sequence
    while n.Pred ≠ null do                  ▷ Until we reach the root
        π ← n.Action ∘ π                    ▷ Prepend action to path
        n ← n.Pred                          ▷ Move to parent node
    end while
    return π                                ▷ Return the action sequence
end function
```

Visual:



Solution: $\pi = (a_1, a_2, a_3)$

3 Algorithm Evaluation Criteria

Completeness

Question: Is the algorithm guaranteed to find a solution when one exists?
Importance: We don't want an algorithm that might miss solutions!

Optimality

Question: Is the returned solution guaranteed to be optimal (minimum cost)?
Importance: Often we don't just want *any* solution, we want the *best* one!

Time Complexity

Measurement: Number of states generated (or nodes expanded)
Typical parameters:

- b = **branching factor** (average number of successors per state)
- d = **goal depth** (minimum depth of any goal node)
- m = **maximum depth** of the state space

Space Complexity

Measurement: Maximum number of nodes stored in memory simultaneously
Why it matters: Memory is often the limiting factor in search!

- Time: If slow, we can wait
- Space: If too much, the program crashes!

4 Depth-First Search (DFS)

DFS Strategy

Core Idea: Go as deep as possible, then backtrack when stuck
Data Structure: LIFO queue (stack) - Last In, First Out
Behavior: Explores one branch completely before trying another

4.1 DFS Algorithm (Tree Search)

Depth-First Search - Tree Version

```
1:  function DFS
2:      n ← InitNode()                          ▷ Create root node from initial state
3:      Q ← Empty LIFO queue (stack)            ▷ Initialize stack for open list
4:      Push(Q, n)                              ▷ Add initial node to stack
5:      while Q is not empty do                 ▷ Continue while there are unexplored nodes
6:          n ← Pop(Q)                          ▷ Take MOST RECENT node (LIFO!)
7:          if GoalTest(n.State) then           ▷ Check if we found a goal
8:              return ExtractSolution(n)       ▷ Success! Return path
9:          end if
10:         for each a ∈ Actions(n.State) do    ▷ Try all applicable actions
11:             n′ ← SuccNode(n, a)             ▷ Create successor node
12:             Push(Q, n′)                     ▷ Add to TOP of stack
13:         end for
14:     end while
15:     return "unsolvable"                     ▷ No solution exists
16: end function
```

Key Points:

- Line 6: LIFO means the last child added is explored first
- Line 11: New nodes go on top of the stack
- No duplicate checking - same state can be visited multiple times!

4.2 DFS Algorithm (Graph Search)

Depth-First Search - Graph Version with Duplicate Elimination

```
1:  function DFS
2:      n ← InitNode()
3:      Q ← Empty LIFO queue (stack)
4:      Push(Q, n)
5:      closed ← ∅                                  ▷ NEW: Track explored states
6:      while Q is not empty do
7:          n ← Pop(Q)
8:          if GoalTest(n.State) then
9:              return ExtractSolution(n)
10:         end if
11:         closed ← closed ∪ {n.State}             ▷ Mark state as explored
12:         for each a ∈ Actions(n.State) do
13:             n′ ← SuccNode(n, a)
14:             if n′.State ∉ closed then           ▷ Only if not explored yet
15:                 Push(Q, n′)
16:             end if
17:         end for
18:     end while
19:     return "unsolvable"
20: end function
```

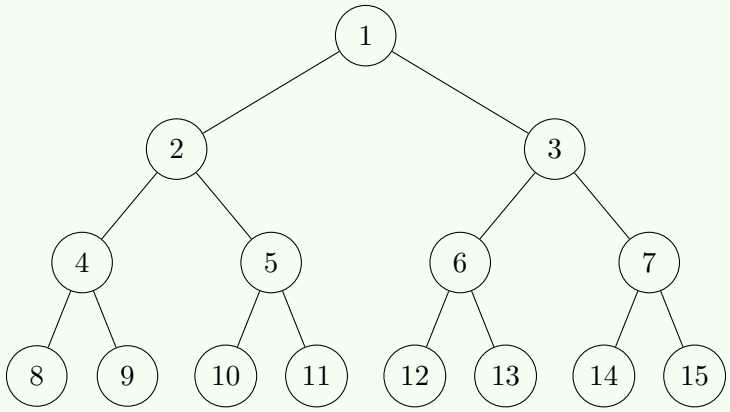
Changes from Tree Search:

- Line 5: Initialize closed list
- Line 11: Add current state to closed after expanding
- Line 14: Check if successor state already explored

4.3 DFS Visual Execution

DFS Execution on a Tree

Consider this tree with branching factor 2:



DFS Expansion Order: 1, 2, 4, 8, 9, 5, 10, 11, 3, 6, 12, 13, 7, 14, 15
Trace:

1. Start: Stack = [1]
2. Expand 1: Stack = [3, 2] (3 on top, 2 below)
3. Expand 2: Stack = [3, 5, 4] (most recent children on top)
4. Expand 4: Stack = [3, 5, 9, 8]
5. Expand 8: Stack = [3, 5, 9] (8 has no children)
6. Expand 9: Stack = [3, 5]
7. Expand 5: Stack = [3, 11, 10]
8. And so on...

Pattern: Always goes to deepest unexplored node before backtracking

4.4 DFS Properties

Property	Value
Complete	No (tree search - can cycle forever) Yes (graph search - closed list prevents cycles)
Optimal	No (finds first solution, not necessarily best)
Time	$O(b^m)$ where m = maximum depth Can be very bad if $m \gg d$
Space	$O(bm)$ (tree search - linear!) $O(b^m)$ (graph search - closed list grows exponentially)

DFS - When to Use

Advantages:

- Low memory (tree version is linear)
- Good when solutions are deep
- Simple to implement

Disadvantages:

- Not complete (tree version)
- Not optimal
- Can waste time exploring very deep paths

Best for: Problems where memory is extremely limited and any solution is acceptable

5 Breadth-First Search (BFS)

BFS Strategy

Core Idea: Explore all nodes at depth d before exploring any node at depth $d+1$
Data Structure: FIFO queue - First In, First Out
Behavior: Systematic level-by-level exploration

5.1 BFS Algorithm

Breadth-First Search

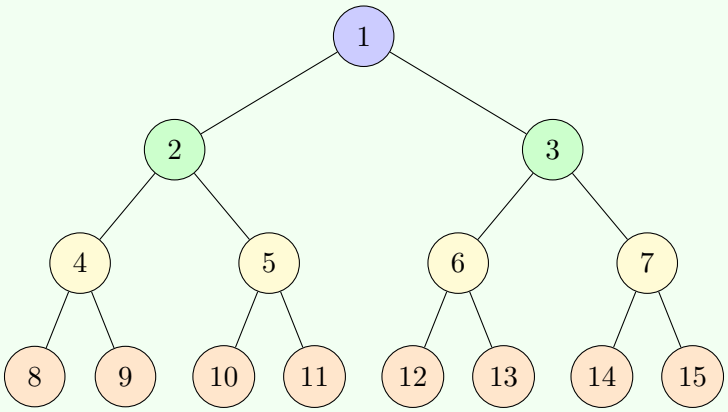
```
1:  function BFS
2:      n ← InitNode()                          ▷ Create root node from initial state
3:      Q ← Empty FIFO queue                    ▷ Initialize queue (not stack!)
4:      Push(Q, n)                              ▷ Add initial node to queue
5:      while Q is not empty do                 ▷ Continue while there are unexplored nodes
6:          n ← Pop(Q)                          ▷ Take OLDEST node (FIFO!)
7:          if GoalTest(n.State) then           ▷ Check if we found a goal
8:              return ExtractSolution(n)       ▷ Success! Return path
9:          end if
10:         for each a ∈ Actions(n.State) do    ▷ Try all applicable actions
11:             n′ ← SuccNode(n, a)             ▷ Create successor node
12:             Push(Q, n′)                     ▷ Add to BACK of queue
13:         end for
14:     end while
15:     return "unsolvable"                     ▷ No solution exists
16: end function
```

Key Difference from DFS:

- Line 6: FIFO means the oldest (earliest added) node is explored first
- This ensures we explore all depth d nodes before any depth $d+1$ nodes

5.2 BFS Visual Execution

BFS Execution on Same Tree



Color Legend:

- Blue = Depth 0
- Green = Depth 1
- Yellow = Depth 2
- Orange = Depth 3

BFS Expansion Order: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
Trace:

1. Start: Queue = [1]
2. Expand 1: Queue = [2, 3]
3. Expand 2: Queue = [3, 4, 5]
4. Expand 3: Queue = [4, 5, 6, 7]
5. Expand 4: Queue = [5, 6, 7, 8, 9]
6. Expand 5: Queue = [6, 7, 8, 9, 10, 11]
7. And so on...

Pattern: All nodes at depth d are expanded before any node at depth $d + 1$

5.3 BFS Properties

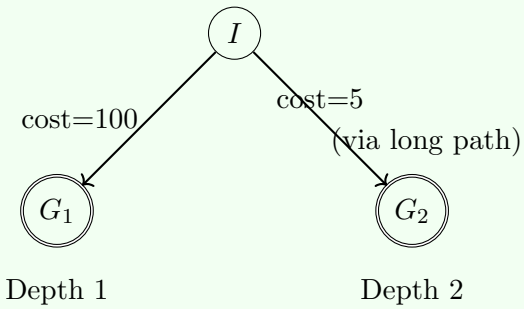
Property	Value
Complete	Yes (will find shallowest goal)
Optimal	Only for unit cost If all actions have cost 1, BFS finds optimal solution If actions have different costs, BFS is NOT optimal
Time	$O(b^d)$ where d = goal depth
Space	$O(b^d)$ (exponential - can be huge!)

Why BFS is Optimal for Unit Cost

If all actions cost 1, then:

- Path cost = number of actions = depth
- BFS finds shallowest goal first
- Therefore, BFS finds minimum-cost goal first
- Hence, BFS is optimal!

But if actions have different costs:



BFS would find G_1 first (depth 1), but G_2 is cheaper (cost 5 vs 100)!

BFS - When to Use

Advantages:

- Complete
- Optimal for unit cost
- Systematic exploration

Disadvantages:

- High memory (exponential in depth)
- Not optimal for non-unit costs

Best for: Unit-cost problems where memory is not a constraint

6 Uniform-Cost Search (UCS)

UCS Strategy

Core Idea: Always expand the node with lowest path cost $g(n)$
Data Structure: Priority queue ordered by g-value
Key Insight: UCS is **equivalent to Dijkstra's algorithm** on the state space graph!
Why it's optimal: We always expand cheapest first, so when we reach a goal, we know we found the cheapest path

6.1 UCS Algorithm (Graph Search)

Uniform-Cost Search - The Optimal Algorithm

```
1:  function UniformCostSearch
2:      n ← InitNode()                                          ▷ Create root node
3:      open ← Empty priority queue ordered by g                ▷ Priority by cost!
4:      closed ← ∅                                              ▷ Track explored states
5:      Insert n into open                                      ▷ Add initial node with g = 0
6:      while open is not empty do                              ▷ Continue while unexplored nodes exist
7:          n ← Pop(open)                                       ▷ Get node with LOWEST g(n)
8:          if GoalTest(n.State) then                           ▷ Check if goal
9:              return ExtractSolution(n)                       ▷ Found optimal solution!
10:         end if
11:         closed ← closed ∪ {n.State}                         ▷ Mark as explored
12:         for each a ∈ Actions(n.State) do                    ▷ Try all actions
13:             n′ ← SuccNode(n, a)                             ▷ Create successor with updated cost
14:             if n′.State ∉ closed and n′.State ∉ States(open) then
15:                 Insert n′ into open                         ▷ New state - add to open
16:             else if ∃n″ ∈ open s.t. n″.State = n′.State and g(n′) < g(n″) then
17:                 Replace n″ with n′ in open                  ▷ Found cheaper path - update!
18:             end if
19:         end for
20:     end while
21:     return "unsolvable"                                     ▷ No solution exists
22: end function
```

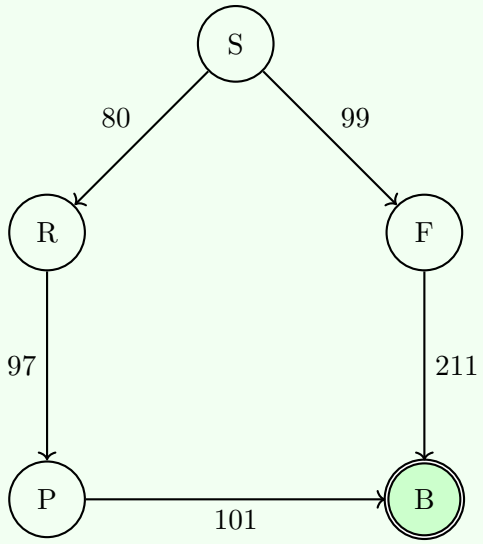
Critical Lines Explained:

- Line 7: Always expand the cheapest node - this is why UCS is optimal!
- Line 14-15: If we've never seen this state, add it to open list
- Line 16-17: **Re-opening**: If we found a cheaper path to a state already in open, update it!
- **Important**: We never re-open closed states (once optimal path found, it stays optimal)

6.2 UCS Visual Example - Romania Routing

UCS Step-by-Step Execution

Find cheapest path from Sibiu (S) to Bucharest (B):



Execution Trace:

Step	Expand	g-value	Open List (state: g-value)
1	S	0	R:80, F:99
2	R	80	F:99, P:177
3	F	99	P:177, B:310
4	P	177	B:278 (updated from 310!)
5	B	278	GOAL FOUND!

Key Moment - Step 4:

- When we expand P, we find path S→R→P→B with cost 278
- B is already in open with cost 310 (from S→F→B)
- We update B's g-value to 278 (cheaper path!)
- This is why line 16-17 of the algorithm is crucial

Solution: S → R → P → B with cost 278

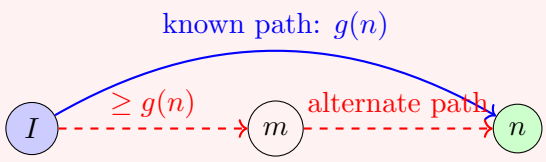
6.3 Why UCS is Optimal - Proof Intuition

Optimality of UCS

Theorem: UCS always finds an optimal solution (if one exists)
Intuition:

1. UCS always expands nodes in order of increasing g-value
2. When we expand a node n, we have found the optimal path to n
3. Why? Any unexplored path to n must go through an unexplored node m
4. But $g(m) \geq g(n)$ (otherwise we'd expand m first)
5. So any path through m costs at least $g(m) \geq g(n)$
6. Therefore, when we expand a goal node, we've found the optimal solution!

Visual:



m not yet expanded: $g(m) \geq g(n)$ n expanded with cost $g(n)$

6.4 UCS Properties

Property	Value
Complete	Yes (will eventually expand all reachable states)
Optimal	**YES - ALWAYS!** Works for any non-negative action costs
Time	$O(b^{1+\lfloor g^*/\epsilon \rfloor})$ where ϵ = minimum positive action cost Intuition: explores all states with $g < g^*$
Space	$O(b^{1+\lfloor g^*/\epsilon \rfloor})$ (exponential) Need to store open + closed lists

UCS - When to Use

Advantages:

- **Always optimal** - guaranteed to find cheapest solution
- Complete
- Works with any non-negative action costs

Disadvantages:

- High memory (exponential space)
- Can explore many states before finding goal
- Requires priority queue (more overhead than simple queue/stack)

Best for: When you **need optimality** with non-unit costs

7 Iterative Deepening Search (IDS)

IDS Strategy

Core Idea: Run DFS with depth limit 0, then 1, then 2, etc., until goal found
Purpose: Combines completeness + optimality (unit cost) of BFS with low memory of DFS
Key Insight: Regenerating states multiple times is worth it for the memory savings!

7.1 IDS Algorithm

Iterative Deepening - Main Function

```
function IterativeDeepening
    n ← InitNode()                                  ▷ Create root node once
    for depth = 0 to ∞ do                           ▷ Try increasing depth limits
        result ← DFSLimited(n, depth)               ▷ Run depth-limited DFS
        if result ≠ "cutoff" then                   ▷ If found goal or proved unsolvable
            return result                           ▷ Return the solution!
        end if
    end for
end function
```

DFSLimited - Depth-Limited DFS

```
function DFSLimited(n, depth)
    if GoalTest(n.State) then                       ▷ Check goal before depth limit
        return ExtractSolution(n)                   ▷ Success!
    end if
    if depth = 0 then                               ▷ Reached depth limit?
        return "cutoff"                             ▷ Can't go deeper - return cutoff
    end if
    cutoffOccurred ← false                          ▷ Track if any child hit limit
    for each a ∈ Actions(n.State) do                ▷ Try all actions
        n′ ← SuccNode(n, a)                         ▷ Create successor
        result ← DFSLimited(n′, depth −1)           ▷ Recursive call, depth-1
        if result = "cutoff" then                   ▷ Child hit depth limit
            cutoffOccurred ← true                   ▷ Remember this
        else if result ≠ "unsolvable" then          ▷ Found solution!
            return result                           ▷ Propagate solution up
        end if
    end for
    if cutoffOccurred then                          ▷ Did any child hit limit?
        return "cutoff"                             ▷ Yes - might find deeper
    else
        return "unsolvable"                         ▷ No - truly no solution this branch
    end if
end function
```

Return Values Explained:

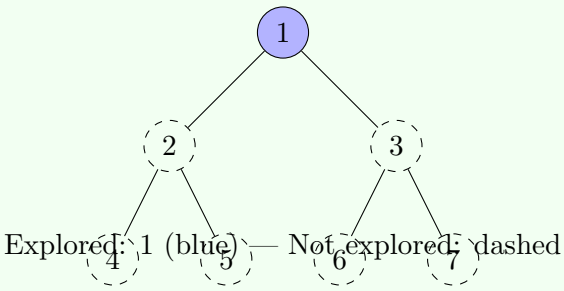
- **Solution**: Found a goal! Return the action path
- **"cutoff"**: Hit depth limit, solution might exist deeper

- **"unsolvable"**: Explored fully, no solution in this subtree

7.2 IDS Visual Execution

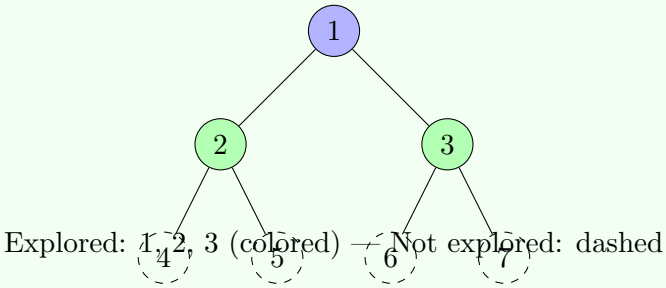
IDS Progressive Deepening

Depth Limit = 0: Only check root



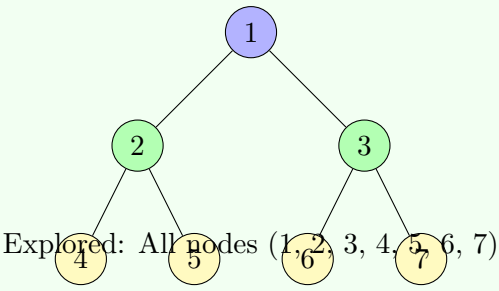
Explored: 1 (blue) — Not explored: dashed

Depth Limit = 1: Check root + children



Explored: 1, 2, 3 (colored) — Not explored: dashed

Depth Limit = 2: Check root + children + grandchildren



Explored: All nodes (1, 2, 3, 4, 5, 6, 7)

Pattern: Each iteration re-explores shallower nodes, but memory is freed between iterations!

7.3 IDS Time Complexity Analysis

Why IDS is Not Wasteful

Concern: Don't we waste time regenerating nodes?
Answer: Not really! Most work is at the deepest level.
Analysis: For goal at depth d with branching factor b:
BFS: Generate $b + b^2 + b^3 + \cdots + b^d$ nodes
IDS:

- Depth 0: Generate 1 node (root)
- Depth 1: Generate $1 + b$ nodes
- Depth 2: Generate $1 + b + b^2$ nodes
- ...
- Depth d: Generate $1 + b + b^2 + \cdots + b^d$ nodes

Total for IDS: $d \cdot 1 + (d-1) \cdot b + (d-2) \cdot b^2 + \cdots + 2 \cdot b^{d-1} + 1 \cdot b^d$
Concrete Example ($b = 10$, $d = 5$):

$$\text{BFS} = 10 + 100 + 1000 + 10000 + 100000 = 111,110$$
$$\text{IDS} = 50 + 400 + 3000 + 20000 + 100000 = 123,450$$

Overhead: Only $\frac{123450-111110}{111110} = 11\%$ more work!
But: Exponentially less memory!

7.4 IDS Properties

Property	**Value**
Complete	Yes (will find solution at depth d)
Optimal	Only for unit cost Like BFS, finds shallowest goal Shallowest = cheapest only if unit cost
Time	$O(b^d)$ (same as BFS asymptotically)
Space	$O(bd)$ **LINEAR!** Only stores current path + siblings

IDS - When to Use

Advantages:

- **Best of both worlds**: BFS completeness/optimality + DFS memory
- Linear space $O(bd)$ - huge advantage!
- Complete and optimal for unit cost

Disadvantages:

- Regenerates nodes (but overhead is small)
- Not optimal for non-unit costs
- Multiple iterations take time

Best for: Unit-cost problems with **large state spaces** where memory is limited

8 Algorithm Comparison Summary

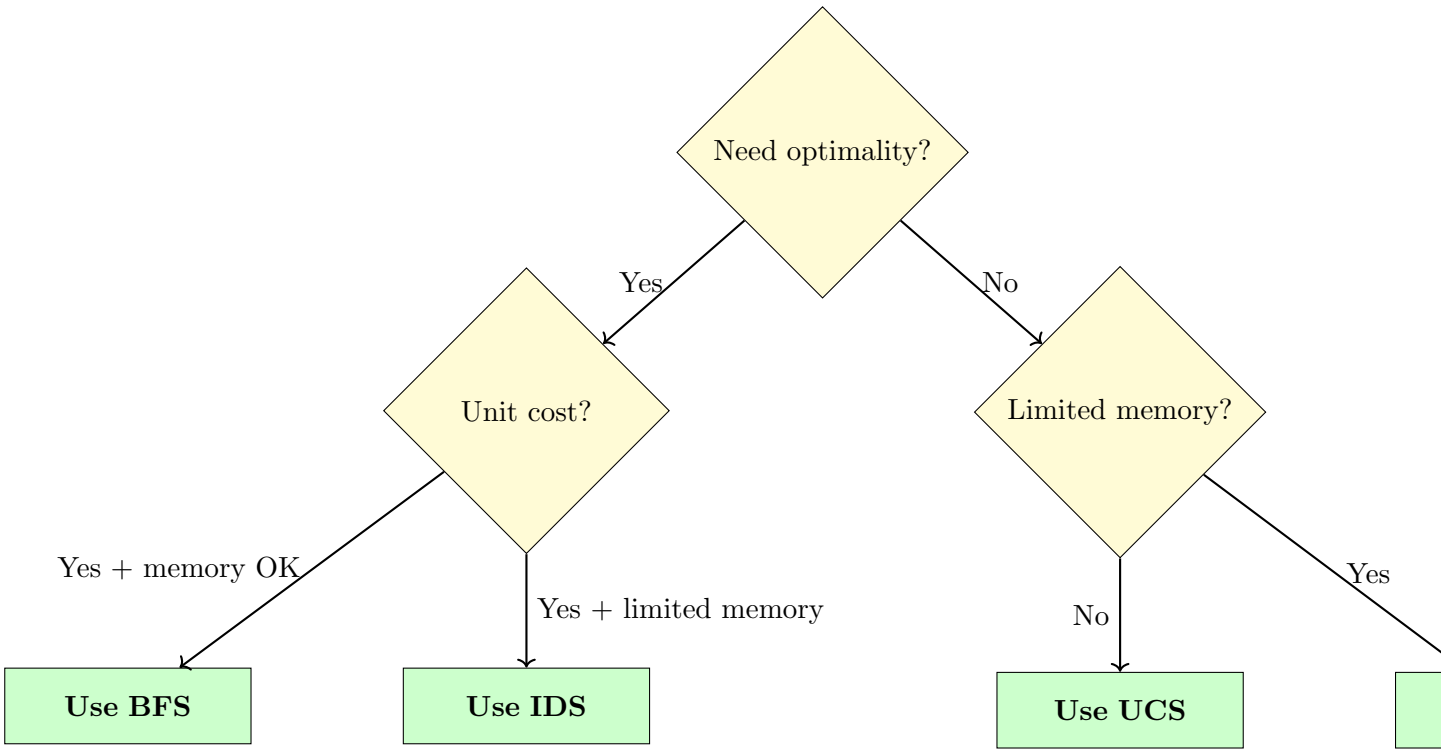
Algorithm	Complete?	Optimal?	Time	Space
DFS	No* Yes**	No	$O(b^m)$	$O(bm)$ $O(b^m)$**
BFS	Yes	Unit cost	$O(b^d)$	$O(b^d)$
UCS	Yes	**Yes**	$O(b^{1+\lfloor g^*/\epsilon \rfloor})$	$O(b^{1+\lfloor g^*/\epsilon \rfloor})$
IDS	Yes	Unit cost	$O(b^d)$	$O(bd)$

Table 1: * Tree search — ** Graph search

Parameters:

- b = branching factor
- d = goal depth (minimum depth of goal)
- m = maximum depth
- g^* = optimal solution cost
- ϵ = minimum positive action cost

8.1 Decision Guide



Quick Selection Guide

Use UCS when: Need optimal solution with non-unit costs
Use IDS when: Need optimal solution with unit costs and limited memory
Use BFS when: Need optimal solution with unit costs and memory is OK
Use DFS when: Don't need optimality, just need *any* solution, very limited memory

9 Tree vs Graph Search

Duplicate Elimination

Tree Search: No duplicate checking

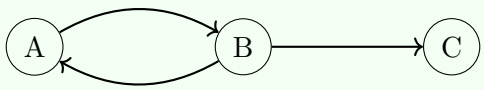
- Same state can appear in multiple nodes
- Lower memory (no closed list)
- Can cycle infinitely
- DFS tree search is NOT complete

Graph Search: With duplicate elimination

- Maintain closed list of explored states
- Check each generated state against closed list
- Discard if already explored
- Higher memory but prevents cycles
- Each state expanded at most once

Visual: Tree vs Graph Search

Consider this state space with a cycle:



Tree Search: A → B → A → B → A → B → ... (infinite loop!)
Graph Search:

1. Expand A, add A to closed
2. Generate B, add to open
3. Expand B, add B to closed
4. Generate A (from B) - already in closed, discard!
5. Generate C, add to open
6. Expand C...

Result: No infinite loop!

10 Practical Problem Solving

10.1 Problem Formulation Steps

When solving a problem with state space search:

1. **Choose state representation**

- What variables describe a configuration?
- Example: $(truck_location, package_location)$

2. **Define initial state**
   - What is the starting configuration?
   - Example: $(truck = A, package = C)$
3. **Define goal test**
   - How do we recognize a goal?
   - Example: $package = D$ (any truck location)
4. **Define actions**
   - What can we do in each state?
   - Example: drive, load, unload
5. **Define transition model**
   - How do actions change states?
   - Example: $drive_{AB}$ changes $(t = A, p)$ to $(t = B, p)$
6. **Define costs**
   - What does each action cost?
   - Example: drive costs distance, load/unload cost 1

10.2 Example: Taxi Problem

Complete Taxi Problem Formulation

Problem: Taxi at location A, two passengers at C and E. Transport one passenger to F.
State: (t, p_1, p_2) where:

- $t \in \{A, B, C, D, E, F\}$ = taxi location
- $p_i \in \{A, B, C, D, E, F, t\}$ = passenger location (t means "in taxi")

Initial: $(t = A, p_1 = C, p_2 = E)$
Goal: $(t = ?, p_1 = F, p_2 = ?)$ or $(t = ?, p_1 = ?, p_2 = F)$
Actions:

- $drive_{XY}$: move taxi from X to Y (cost = distance)
- $getin_{iX}$: passenger i boards at X (cost = 1)
- $getout_{iX}$: passenger i exits at X (cost = 1)

Example solution:

1. $drive_{AB}$: $(A, C, E) \to (B, C, E)$
2. $drive_{BC}$: $(B, C, E) \to (C, C, E)$
3. $getin_1C$: $(C, C, E) \to (C, t, E)$
4. $drive_{CF}$: $(C, t, E) \to (F, t, E)$
5. $getout_1F$: $(F, t, E) \to (F, F, E)$ - GOAL!

11 Key Insights and Common Mistakes

Critical Facts to Remember

1. **UCS is ALWAYS optimal** (for any non-negative costs)
2. **BFS and IDS are optimal ONLY for unit cost**
3. **IDS has best space complexity** ($O(bd)$ - linear!)
4. **DFS (tree) is NOT complete** (can cycle forever)
5. **Graph search prevents cycles** (use closed list)
6. **UCS is equivalent to Dijkstra's algorithm**

Common Mistakes

1. Using BFS for non-unit costs - It's not optimal!
2. Forgetting duplicate elimination in DFS - Can loop forever!
3. Not updating g-values in UCS - Must handle re-opening!
4. Confusing open and closed lists
   - Open = candidates for expansion
   - Closed = already expanded
5. Wrong goal test timing
   - Check when expanding (as shown in algorithms)
   - Could also check when generating for early termination

12 Formulas to Memorize

Essential Formulas

Path Cost:

$$\text{cost}(\pi) = \sum_{i=1}^{n} \text{cost}(a_i)$$

DFS Complexity:

- Time: $O(b^m)$
- Space: $O(bm)$ tree, $O(b^m)$ graph

BFS Complexity:

- Time: $O(b^d)$
- Space: $O(b^d)$

UCS Complexity:

- Time: $O(b^{1+\lfloor g^*/\epsilon \rfloor})$
- Space: $O(b^{1+\lfloor g^*/\epsilon \rfloor})$

IDS Complexity:

- Time: $O(b^d)$
- Space: $O(bd)$ LINEAR!

13 Final Exam Tips

1. **When tracing algorithms**:
   - Show open/closed lists at each step
   - For UCS, always show g-values
   - Be consistent with tie-breaking rules
2. **When formulating problems**:
   - Clearly define state representation
   - List all actions with preconditions
   - Specify action costs
3. **When comparing algorithms**:
   - Consider completeness, optimality, time, space
   - Remember unit vs non-unit cost distinction
   - Think about memory constraints